

Modül 30

Mimari Best Practices

Eğitimin Final Modülü

Modül:	30 / 30
Ders Sayısı:	8 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Final Modüle Hoş Geldin

Tebrikler! 30 modüllük yolculuğun son durağındasın. Bu modülde tüm öğrendiklerini birleştirip büyük projeler için mimari kararlar almayı öğreneceğiz. TSE'deki Flexi.Kalite, APM gibi enterprise projelerin altyapısını tasarlamak için ihtiyacın olan her şey burada.

Öğrenecekler:

- ✓ Folder structure ve modülerizasyon
- ✓ Feature-Based Architecture
- ✓ Layered Architecture (Domain/Application/Infrastructure)
- ✓ Smart vs Dumb Components
- ✓ State management stratejileri
- ✓ Error handling stratejisi
- ✓ Code splitting ve performance
- ✓ Team conventions ve standards

Folder Structure

Büyük projeler için klasör organizasyonu.

☐ Feature-Based Structure (Önerilen)

```

src/
├── app/
│   ├── core/                                # Tek instance services
│   │   ├── auth/
│   │   ├── http/
│   │   └── interceptors/
│   ├── shared/                             # Birden fazla feature kullanır
│   │   ├── components/                     # Reusable UI
│   │   ├── directives/
│   │   ├── pipes/
│   │   └── models/
│   ├── features/                          # Her feature kendi modülü
│   │   ├── users/
│   │   │   ├── pages/
│   │   │   │   ├── user-list.page.ts
│   │   │   │   └── user-detail.page.ts
│   │   │   ├── components/
│   │   │   │   ├── user-card.component.ts
│   │   │   │   └── user-form.component.ts
│   │   │   ├── services/
│   │   │   │   ├── user-api.service.ts
│   │   │   │   └── user-store.service.ts
│   │   │   ├── models/
│   │   │   │   └── user.model.ts
│   │   │   └── users.routes.ts
│   │   ├── products/
│   │   └── orders/
│   ├── layouts/                           # Layout component'leri
│   │   ├── main-layout/
│   │   └── auth-layout/
│   ├── app.config.ts
│   ├── app.routes.ts
│   └── app.component.ts
├── environments/
│   ├── environment.ts
│   └── environment.prod.ts
└── styles/                                # Global stiller
    ├── _variables.scss
    └── styles.scss

```

□ Kurallar

- ✓ Core sadece tek instance service'ler için (auth, logging)
- ✓ Shared reusable component/pipe/directive'ler
- ✓ Features bağımsız iş alanları (users, products, ...)

- ✓ Feature → Feature direkt import etme — shared'a koy
- ✓ Pages route'a bağlı, container component'ler
- ✓ Components presentational, reusable

Feature-Based Architecture

Her feature kendi başına bir küçük uygulama.

□ Feature Yapısı

```
features/users/
├── pages/                                # Route'lara bağlı container'lar
│   ├── user-list.page.ts                # Smart component
│   └── user-detail.page.ts
├── components/                          # Dumb components
│   ├── user-card.component.ts
│   ├── user-form.component.ts
│   └── user-avatar.component.ts
├── services/
│   ├── user-api.service.ts             # HTTP API
│   └── user-store.service.ts           # State management
├── models/
│   ├── user.model.ts
│   └── user.dto.ts
├── guards/
│   └── user-edit.guard.ts
├── resolvers/
│   └── user.resolver.ts
└── users.routes.ts                     # Feature routes
```

□ Feature Routes

```
// features/users/users.routes.ts
import { Routes } from '@angular/router';

export const USERS_ROUTES: Routes = [
  {
    path: '',
    loadComponent: () => import('./pages/user-list.page').then(m => m.UserListPage)
  },
  {
    path: ':id',
    loadComponent: () => import('./pages/user-detail.page').then(m =>
m.UserDetailPage),
    canActivate: [authGuard],
    resolve: { user: userResolver }
  },
  {
    path: ':id/edit',
    loadComponent: () => import('./pages/user-edit.page').then(m => m.UserEditPage),
    canActivate: [authGuard, permissionGuard(['users:write'])],
    canDeactivate: [unsavedChangesGuard]
  }
];

// app.routes.ts
export const routes: Routes = [
  {
    path: 'users',
    loadChildren: () => import('./features/users/users.routes').then(m =>
m.USERS_ROUTES)
  }
];
```

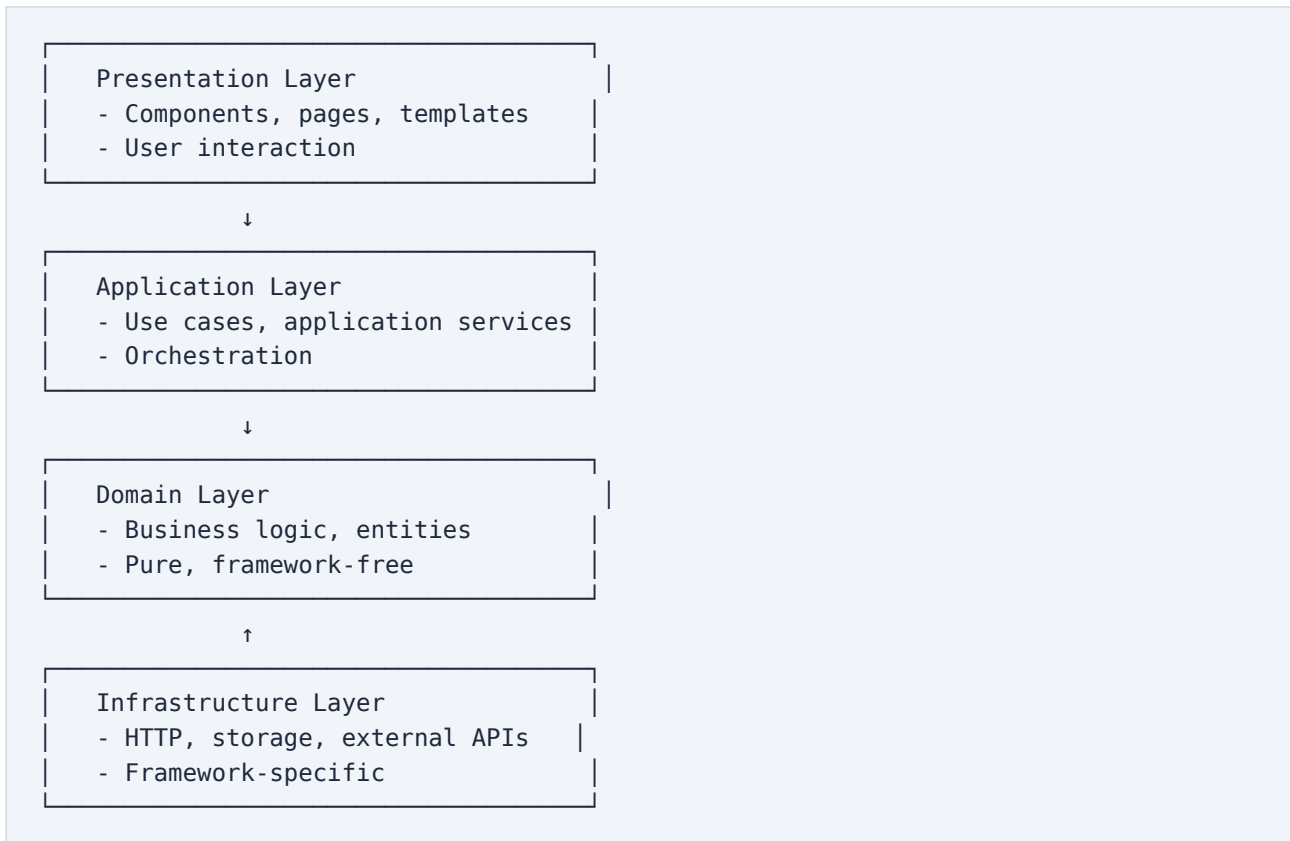
□ Faydaları

- ✓ Bağımsız geliştirme — takımlar paralel çalışabilir
- ✓ Lazy loading kolay — feature başına chunk
- ✓ Refactoring kolay — bir feature'ı taşımak/silmek mümkün
- ✓ Test izolasyonu — feature'lar bağımsız test
- ✓ Code ownership — her feature bir takıma

Layered Architecture

Domain / Application / Infrastructure ayrımı.

□ Katmanlar



□ Domain Layer


```
// domain/user/user.entity.ts
// Framework'ten bağımsız iş mantığı
export class User {
  constructor(
    public readonly id: string,
    private _email: string,
    private _role: UserRole
  ) {}

  get email(): string { return this._email; }
  get role(): UserRole { return this._role; }

  changeEmail(newEmail: string): void {
    if (!this.isValidEmail(newEmail)) {
      throw new Error('Invalid email');
    }
    this._email = newEmail;
  }

  canEdit(resource: { ownerId: string }): boolean {
    return this.role === 'admin' || this.id === resource.ownerId;
  }

  private isValidEmail(email: string): boolean {
    return /^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$/.test(email);
  }
}
```

⚙ Application Layer

```
// application/user/use-cases/update-user-email.ts
@Injectable({ providedIn: 'root' })
export class UpdateUserEmailUseCase {
  private userRepo = inject(UserRepository);
  private notifier = inject(NotificationService);
  private logger = inject(LoggerService);

  async execute(userId: string, newEmail: string): Promise<void> {
    const user = await this.userRepo.findById(userId);
    if (!user) throw new Error('User not found');

    user.changeEmail(newEmail); // Domain logic
    await this.userRepo.save(user);

    await this.notifier.notify(user.id, 'Email updated');
    this.logger.info(`User ${user.id} email changed`);
  }
}
```

▢ Infrastructure Layer

```

// infrastructure/user/user-http.repository.ts
@Injectable({ providedIn: 'root' })
export class UserHttpRepository implements UserRepository {
  private http = inject(HttpClient);

  async findById(id: string): Promise<User | null> {
    try {
      const dto = await firstValueFrom(
        this.http.get<UserDto>(`/api/users/${id}`)
      );
      return this.toDomain(dto); // DTO → Domain mapping
    } catch (err: any) {
      if (err.status === 404) return null;
      throw err;
    }
  }

  async save(user: User): Promise<void> {
    await firstValueFrom(
      this.http.put(`/api/users/${user.id}`, this.toDto(user))
    );
  }

  private toDomain(dto: UserDto): User {
    return new User(dto.id, dto.email, dto.role);
  }

  private toDto(user: User): UserDto {
    return { id: user.id, email: user.email, role: user.role };
  }
}

```

Smart vs Dumb Components

Container/Presentational pattern.

□ Smart Components (Container)

Sorumluluk: State management, side effects, business logic. Genellikle pages/klasöründe.

```

// users/pages/user-list.page.ts
@Component({
  selector: 'app-user-list-page',
  imports: [UserCardComponent, FilterBarComponent],
  template: `
    <app-filter-bar (filterChange)="updateFilter($event)" />

    @if (loading()) {
      <app-spinner />
    } @else {
      @for (user of filteredUsers(); track user.id) {
        <app-user-card
          [user]="user"
          (edit)="onEdit($event)"
          (delete)="onDelete($event)">
        </app-user-card>
      }
    }
  `
})
export class UserListPage {
  private store = inject(UserStore);
  private router = inject(Router);

  // State
  users = this.store.users;
  loading = this.store.loading;
  filter = signal<UserFilter>({});

  filteredUsers = computed(() =>
    this.users().filter(u => matchesFilter(u, this.filter()))
  );

  // Business logic
  updateFilter(filter: UserFilter) {
    this.filter.set(filter);
  }

  onEdit(user: User) {
    this.router.navigate([' /users', user.id, 'edit']);
  }

  async onDelete(user: User) {
    await this.store.deleteUser(user.id);
  }
}

```

❑ Dumb Components (Presentational)

Sorumluluk: Sadece görsel. Input alır, output emit eder. Reusable ve test edilebilir.

```
// users/components/user-card.component.ts
@Component({
  selector: 'app-user-card',
  template: `
    <div class="card">
      <img [src]="user().avatar" alt="Avatar">
      <h3>{{ user().name }}</h3>
      <p>{{ user().email }}</p>

      @if (canEdit()) {
        <button (click)="edit.emit(user())">Düzenle</button>
      }
      @if (canDelete()) {
        <button (click)="delete.emit(user())">Sil</button>
      }
    </div>
  `,
  changeDetection: ChangeDetectionStrategy.OnPush
})
export class UserCardComponent {
  user = input.required<User>();
  canEdit = input(true);
  canDelete = input(true);

  edit = output<User>();
  delete = output<User>();
}
```

⚖ Karşılaştırma

Konu	Smart	Dumb
Data	Service'ten alır	Input olarak alır
Side effects	Var	Yok
Reusability	Düşük	Yüksek
Test	Mock'lar gerek	Pure, kolay test
Konum	pages/	components/

State Management Stratejisi

Hangi state'i nerede yöneteceğine karar vermek.

State Türleri

State Türü	Yer	Örnek
Local UI	Component signal	Modal açık mı
Feature	Feature service	Aktif filter
Global	Global store	Logged in user
Server	Cache service	API response
URL	Router params	Sayfa numarası
Persistent	localStorage	Theme tercihi

Karar Ağacı

State Nereye Konulur?

Bir component kullanıyor? → Component signal
Birkaç sibling kullanıyor? → Feature service
Tüm app kullanıyor? → Global store
Server'dan geliyor? → httpResource + cache
Sayfa açıldığında olmalı? → URL/route params
Reload sonrası kalmalı? → localStorage

Modern Signal Store Pattern

```

// features/users/services/user-store.service.ts
@Injectable({ providedIn: 'root' })
export class UserStore {
  private api = inject(UserApiService);

  // Private writable
  private _users = signal<User[]>([]);
  private _selectedId = signal<string | null>(null);
  private _loading = signal(false);

  // Public readonly
  readonly users = this._users.asReadonly();
  readonly loading = this._loading.asReadonly();
  readonly selectedId = this._selectedId.asReadonly();

  // Computed
  readonly selected = computed(() =>
    this._users().find(u => u.id === this._selectedId())
  );

  readonly userCount = computed(() => this._users().length);

  // Actions
  async loadUsers() {
    this._loading.set(true);
    try {
      const users = await firstValueFrom(this.api.getUsers());
      this._users.set(users);
    } finally {
      this._loading.set(false);
    }
  }

  selectUser(id: string) {
    this._selectedId.set(id);
  }

  async deleteUser(id: string) {
    await firstValueFrom(this.api.deleteUser(id));
    this._users.update(users => users.filter(u => u.id !== id));
  }
}

```

Error Handling Stratejisi

Hataları katmanlar arasında nasıl yöneteceğiz?

3 Katmanlı Error Handling

1. Network Layer (Interceptor)
 - Global error logging
 - 401 → refresh token
 - 5xx → retry

↓

2. Service Layer
 - Domain-specific errors
 - Error normalization

↓

3. Component Layer
 - User feedback (toast)
 - UI state (error state)

Global Error Handler


```
@Injectable()
export class GlobalErrorHandler implements ErrorHandler {
  private logger = inject(LoggerService);
  private toast = inject(MessageService);
  private zone = inject(NgZone);

  handleError(error: any): void {
    this.logger.error('Global error', error);

    // Sentry'ye gönder
    Sentry.captureException(error);

    this.zone.run(() => {
      this.toast.add({
        severity: 'error',
        summary: 'Beklenmeyen hata',
        detail: 'Bir sorun oluştu. Lütfen tekrar deneyin.'
      });
    });
  }
}

// app.config.ts
providers: [
  { provide: ErrorHandler, useClass: GlobalErrorHandler }
]
```

❑ Result Pattern (Functional)

```
// Hata tipini explicit yap
type Result<T, E = Error> = Success<T> | Failure<E>;

interface Success<T> { ok: true; value: T; }
interface Failure<E> { ok: false; error: E; }

function ok<T>(value: T): Success<T> { return { ok: true, value }; }
function err<E>(error: E): Failure<E> { return { ok: false, error }; }

// Service
async function getUser(id: string): Promise<Result<User, ApiError>> {
  try {
    const user = await fetchUser(id);
    return ok(user);
  } catch (e: any) {
    return err({ code: e.code, message: e.message });
  }
}

// Component
const result = await getUser('123');
if (result.ok) {
  console.log(result.value); // User
} else {
  console.log(result.error); // ApiError
}
```

Performance Strategy

Büyük app'lerin hızlı kalması için.

⚡ Performance Checklist

- ✓ OnPush change detection tüm component'lerde
- ✓ Signal-based state kullan (otomatik OnPush)
- ✓ Lazy load tüm feature route'ları
- ✓ @defer below-the-fold content
- ✓ track kullan @for içinde
- ✓ computed() ile memoization
- ✓ Virtual scroll 100+ item için
- ✓ NgOptimizedImage tüm image'larda
- ✓ Bundle analysis her release'de
- ✓ Zoneless deneme (modern projeler)

📦 Performance Budget

```
// angular.json
{
  "configurations": {
    "production": {
      "budgets": [
        {
          "type": "initial",
          "maximumWarning": "500kb",
          "maximumError": "1mb"
        },
        {
          "type": "anyComponentStyle",
          "maximumWarning": "2kb",
          "maximumError": "4kb"
        }
      ]
    }
  }
}
```

📦 Code Splitting Strategy

```
// Feature başına chunk
{
  path: 'users',
  loadChildren: () => import('./features/users/users.routes')
}

// Heavy library lazy
@defer (on viewport) {
  <app-chart [data]="data()" /> // Chart.js sadece görünce yüklenir
}

// Conditional import
async loadHeavyFeature() {
  const { HeavyService } = await import('./heavy.service');
  return new HeavyService();
}
```

Team Conventions & Standards

Büyük takımlarda tutarlılık için.

□ Naming Conventions

Type	Convention	Örnek
Component	kebab-case	user-card.component.ts
Service	kebab-case	user-api.service.ts
Class	PascalCase	UserApiService
Interface	PascalCase	User, CreateUserDto
Function	camelCase	getUserById
Constant	UPPER_SNAKE	MAX_RETRIES
Selector	app-kebab	<app-user-card>

□ Tooling

```
// .eslintrc.json
{
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:@angular-eslint/recommended"
  ],
  "rules": {
    "@angular-eslint/component-selector": ["error", { "prefix": "app", "style": "kebab-case" }],
    "@angular-eslint/no-empty-lifecycle-method": "error",
    "@typescript-eslint/no-explicit-any": "warn"
  }
}

// .prettierrc
{
  "singleQuote": true,
  "trailingComma": "es5",
  "printWidth": 100,
  "tabWidth": 2
}

// Husky pre-commit
{
  "husky": {
    "hooks": {
      "pre-commit": "lint-staged"
    }
  },
  "lint-staged": {
    "*.{ts,html}": ["eslint --fix", "prettier --write"]
  }
}
```

□ Documentation Standards

```
/**
 * Kullanıcı bilgilerini getirir.
 *
 * @param id - Kullanıcının unique ID'si
 * @returns User objesi, bulunamazsa null
 * @throws {ApiError} 401 yetkisiz erişim
 *
 * @example
 * const user = await getUser('user-123');
 * if (user) console.log(user.name);
 */
async function getUser(id: string): Promise<User | null> {
  // ...
}
```

□ Pull Request Standards

- ✓ Title: "feat:", "fix:", "refactor:" prefix'leri
- ✓ Description: Ne, neden, nasıl test edildi
- ✓ Screenshots: UI değışiklik varsa
- ✓ Tests: Yeni feature için test eklendi mi
- ✓ Breaking changes: Açıkça belirt
- ✓ Reviewer: En az 1 onay zorunlu

📦 Eğitim Tamamlandı!

Tebrikler Emre! 30 modüllük Angular v21 yolculuğunu başarıyla tamamladın. Bu eğitim boyunca yaklaşık 200+ ders ve onlarca pattern öğrendin. Artık modern Angular ile profesyonel, scalable, maintainable uygulamalar yazabilirsin.

📦 Genel Özet — 30 Modül

Temel Konular (1-4)

- ✓ Angular Felsefe & Modern Yaklaşım
- ✓ Component Temelleri
- ✓ Template Syntax
- ✓ Mini Proje (TodoApp)

Reaktivite & Lifecycle (5-7)

- ✓ Signals & Reactivity Derinlemesine
- ✓ Lifecycle Hooks & Directives
- ✓ Services & Dependency Injection

Routing & Forms (8-9)

- ✓ Routing & Navigation
- ✓ Forms (Reactive, Custom Controls)

HTTP & State (10-11)

- ✓ HttpClient & API Communication
- ✓ Resource API Production Patterns

Templating & Directives (12-14)

- ✓ Pipes & Custom Pipes
- ✓ Directives Derinlemesine
- ✓ State Management

UI & Performance (15-16)

- ✓ Animations
- ✓ Performance Optimization

Testing & Communication (17-18)

- ✓ Testing (Unit, Integration, E2E)
- ✓ Component Communication

Production (19-25)

- ✓ Internationalization (i18n)
- ✓ Server-Side Rendering
- ✓ Progressive Web Apps
- ✓ Security
- ✓ Build & Deployment
- ✓ Micro-Frontend Architecture
- ✓ PrimeNG Derinlemesine

Advanced Topics (26-30)

- ✓ Real-Time Applications
- ✓ File Upload & Download
- ✓ Authentication & Authorization
- ✓ Advanced TypeScript Patterns
- ✓ Mimari Best Practices

□ Bundan Sonrası

- ✓ Pratik yap — küçük projeler ile öğrendiklerini uygula
- ✓ Open source'a katkı — Angular ecosystem'inde

- ✓ Yeni feature'ları takip et — Angular her 6 ayda major release
- ✓ Topluluğa katıl — Discord, Twitter, GitHub Discussions
- ✓ Mentor ol — öğrendiklerini başkalarına anlat
- ✓ Conference talks izle — Angular Connect, ng-conf

▢ Son Söz

Yazılım sürekli değişen bir alan. Modern Angular'ı öğrendin ama sürekli güncel kalman gerekiyor. Önemli olan: temelleri sağlam tutmak, pattern'leri anlamak, ve sürekli öğrenmeye açık olmak.

TSE'deki Flexi.Kalite, APM, FATURA gibi projelerinde başarılar dilerim. Modern, maintainable, scalable kod yaz! ▢